ЛР 3. Твой код простыми словами

Музыкальная клавиатура: как нажатие клавиши в терминале превращается в ноту, где выбирается её длительность и какой код включает звук.

Все пути - внутри `lab3`. Файлы `.c` из таблицы лежат в `Core/Src`, файлы `.h` - в `Core/Inc`. Номера в блоках - строки исходников на 03.11.2026; длинные строки перенесены.

Карта основных файлов

Файл	За что отвечает
`main.c`	Запускает программу и связывает связь, клавиатуру и звук.
`piano.c`, `piano.h`	Выбирают ноту, октаву и длительность. Следят, когда закончить ноту или перейти к следующей в гамме.
`sound.c`, `sound.h`	Настраивают TIM2: включают звуковой сигнал нужной частоты и выключают его.
`clock.c`, `clock.h`	Настраивают частоты работы контроллера и его устройств.
`console.c`, `console.h`	Получают отдельные клавиши, передают их в `piano` и формируют ответы для терминала.
`serial.c`, `serial.h`	Хранят принятые и отправляемые байты в очередях, обслуживают обмен.
`uart_hal.c` `uart_backend.h`	Связывают общий модуль `serial` с USART1 через библиотеку HAL.
`uart_pins.c`	Настраивает выводы связи PA9 и PA10.
`board.h`	Хранит вывод звука PA5, скорость UART и значения частот для проекта.
`gpio_driver.c` `gpio_driver.h`	Твой драйвер настройки выводов. Его используют звук и UART.

Главная цепочка

Клавиша → UART → очередь `serial` → `console_update` → `piano_input` → нужная частота → `sound_set_frequency` → звук на выходе таймера.

В `.h` описаны доступные функции и данные. В `.c` написано, какие действия они выполняют.

1. main.c: порядок работы программы

Что выполняется при запуске

Core/Src/main.c, строки 19-25

```
19      SystemCoreClockUpdate();
20      if (HAL_Init() != HAL_OK) fatal(1);
21      if (!clock_init()) fatal(2);
22      if (!sound_init()) fatal(3);
23      piano_init(&piano);
24      if (!serial_init(true)) fatal(4);
25      console_init(&console);
```

Готовится HAL и время, затем настраиваются частоты (`clock_init`) и звуковой выход (`sound_init`). `piano_init` задаёт исходные настройки и тишину. `serial_init(true)` запускает UART с прерываниями; `console_init` подготавливает вывод сообщений.

В строках **26-30** приветствие с подсказками кладётся в очередь отправки через `serial_write`.

Этот цикл повторяется всё время

Core/Src/main.c, строки 31-36

```
31      for (;;) {
32          serial_service();
33          uint32_t now = HAL_GetTick();
34          piano_update(&piano, now);
35          console_update(&console, &piano, now);
36          if (!sound_set_frequency(piano_frequency_millihz(&piano))) fatal(5);
```

32: продвинуть отправку UART. **33:** взять время в миллисекундах. **34:** проверить, пора ли закончить ноту или сменить ноту гаммы. **35:** обработать новые клавиши. **36:** передать текущую частоту в звуковой модуль.

Когда `piano` возвращает частоту 0, `sound` выключает звук. Сам цикл продолжает проверять время и ввод, пока аппаратный таймер формирует звуковой сигнал.

Где заданы исходные настройки

Настройка	Открой
Семь нот, девять октав, начальная октава 5/9	`Core/Inc/piano.h`, 11-16
Длительность 0,1-5 с, шаг 0,1 с, начальное значение 1 с	`Core/Inc/piano.h`, 17-20
Вывод звука, частота таймера и скорость UART	`Core/Inc/board.h`, 5-10

`SysTick_Handler` (40) обновляет счётчик времени. `fatal` (11-15) сохраняет код ошибки в `lab3_fault` и останавливает выполнение. Код ошибки можно смотреть в отладчике.

2. piano.c: клавиша запускает ноту

Где находится управление клавишами

Клавиша	Действие и строки в piano.c
1-7	Запустить до, ре, ми, фа, соль, ля или си: 70-73.
+ / -	Изменить выбранную октаву в пределах 1-9: 76-87.
A / a	Изменить длительность на 100 мс в пределах 100-5000 мс: 88-99.
Enter	Запустить гамму с ноты до: 100-103.
Другой символ	Вернуть событие ошибки PIANO_EVENT_INVALID: 104-105.

Core/Src/piano.c, строки 70-73

```
70      if (byte >= '1' && byte <= '7') {
71          start_playback(piano, (uint8_t)(byte - '1'), false, now_ms);
72          return PIANO_EVENT_NOTE;
73      }
```

byte - '1' превращает клавишу в номер ноты: 1 даёт 0, 2 даёт 1, ... 7 даёт 6. false означает запуск одной ноты. Для гаммы тот же start_playback вызывается с true в строке 102.

Что запоминается при запуске

Core/Src/piano.c, строки 36-41

```
36      piano->active_octave = piano->selected_octave;
37      piano->active_duration_ms = piano->selected_duration_ms;
38      piano->note_started_at_ms = now_ms;
39      piano->active_note = note;
40      piano->playing = true;
41      piano->scale = scale;
```

Поля selected_* - настройки для следующего запуска. Поля active_* - настройки уже начатого воспроизведения. При старте октава и длительность копируются из выбранных в текущие.

note_started_at_ms хранит время начала ноты; active_note - её номер; playing говорит, что звук должен идти; scale - что нужно продолжать гамму. Описание объекта piano_t находится в Core/Inc/piano.h, строки 39-49.

Пример с изменением настроек

Нажал 6 - играет ля. Затем нажал + и A - выбраны новая октава и длительность. Уже начатая нота доиграет по прежним настройкам. Следующая цифра запустит ноту с новыми. Такой же набор исходных настроек сохраняется на всю запущенную гамму.

3. piano.c: время, гамма и частоты нот

Как заканчивается одна нота

Core/Src/piano.c, строки 115-122

```
115     const uint32_t elapsed_ms = now_ms - piano->note_started_at_ms;
116     if (elapsed_ms < piano->active_duration_ms) {
117         return;
118     }
119     if (!piano->scale) {
120         stop_playback(piano);
121         return;
122     }
```

Код вычитает время начала из текущего времени. Пока длительность не прошла, функция возвращается. Когда время вышло и играет одна нота, вызывается `stop_playback`: сбрасывает `playing`, `scale` и текущую ноту (26-31).

Как гамма переходит к следующей ноте

Core/Src/piano.c, строки 124-132

```
124     const uint32_t notes_elapsed = elapsed_ms / piano->active_duration_ms;
125     const uint32_t notes_remaining = PIANO_NOTE_COUNT - piano->active_note;
126     if (notes_elapsed >= notes_remaining) {
127         stop_playback(piano);
128         return;
129     }
130
131     piano->active_note = (uint8_t)(piano->active_note + notes_elapsed);
132     piano->note_started_at_ms += notes_elapsed * piano->active_duration_ms;
```

`notes_elapsed` - сколько нот уже должно было пройти. Если гамма закончилась, воспроизведение останавливается. Иначе код сдвигает номер ноты и время её начала. При позднем вызове истёкшие ноты пропускаются: время всей гаммы сохраняется.

Где находятся готовые частоты

Массив `note_frequencies` в строках **5-15** содержит девять строк октав и семь столбцов нот. Частоты записаны в миллигерцах: **440000 означает 440 Гц**. Например, строка первой октавы:

Core/Src/piano.c, строка 10

```
10      { 261626u, 293665u, 329628u, 349228u, 391995u, 440000u, 493883u }, /* C4..B4 */
```

Шестое число - частота ля. `piano_note_frequency_millihz` (177-184) выбирает число из таблицы. `piano_frequency_millihz` (135-140) возвращает частоту текущей ноты, а при тишине - 0.

Почему Enter не запускает гамму дважды

Строки **63-68** используют `suppress_lf`: после CR следующий LF игнорируется. Поэтому CRLF считается одним Enter. Отдельный CR или LF запускает гамму через строки 100-103.

4. sound.c: включение и остановка звука

sound_init подготавливает TIM2

Core/Src/sound.c, строки 21-27

```
21      TIM2->CR1 = TIM_CR1_ARPE;
22      TIM2->PSC = 0;
23      TIM2->ARR = 1;
24      TIM2->CCR1 = 0;
25      TIM2->CCMR1 = TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1PE;
26      TIM2->CCER = TIM_CCER_CC1E;
27      TIM2->DIER = 0; /* Waveform generation is entirely in hardware. */
```

Код включает PWM первого канала с начальной тишиной: `CCR1 = 0`. `DIER = 0`: сигнал формирует сам таймер. В строках 32-34 к нему подключается PA5 через AF1.

Как частота ноты превращается в настройку таймера

Core/Src/sound.c, строки 10-12

```
10      if (frequency_millihz < 16000U || frequency_millihz > 8000000U) return 0;
11      return (uint32_t)(((uint64_t)BOARD_TIMER_HZ * 1000U + frequency_millihz / 2U) /
12                        frequency_millihz);
```

`sound_period_ticks` вычисляет число шагов таймера на один период сигнала. Берёт частоту TIM2 из `BOARD_TIMER_HZ`; умножение на 1000 учитывает, что входная частота дана в миллигерцах. Неподходящее значение возвращает 0.

Что делает sound_set_frequency

Core/Src/sound.c, строки 54-59

```
54      uint32_t ticks = sound_period_ticks(frequency_millihz);
55      if (!ticks) return false;
56      if (frequency_millihz == current_frequency) return true;
57      sound_stop();
58      TIM2->ARR = ticks - 1U;
59      TIM2->CCR1 = ticks / 2U;
```

Прежняя частота продолжает звучать без перенастройки. Для новой вычисляются `ticks`, старый звук выключается. ARR получает длину периода минус один, CCR1 - половину периода. В строках **60-64** настройки загружаются и таймер запускается.

Как получается тишина

Core/Src/sound.c, строки 40-43

```
40      TIM2->CCR1 = 0;
41      TIM2->EGR = TIM_EGR_UG;
42      TIM2->CR1 &= ~TIM_CR1_CEN;
43      TIM2->CNT = 0;
```

`sound_stop` задаёт `CCR1 = 0` и останавливает счётчик. На PA5 удерживается низкий уровень. В начале `sound_set_frequency`, строки 50-52, частота 0 вызывает эту остановку.

5. clock.c и board.h: частоты и вывод звука

board.h: значения, которыми пользуется программа

Core/Inc/board.h, строки 5-10

```
 5  #define BOARD_SOUND_PORT GPIOA
 6  #define BOARD_SOUND_PIN 5U
 7  #define BOARD_SOUND_AF 1U
 8  #define BOARD_SYSCLK_HZ 120000000U
 9  #define BOARD_TIMER_HZ 60000000U
10  #define BOARD_UART_BAUD 57600U
```

Звуковой выход - PA5 с AF1. UART работает на скорости 57600. BOARD_TIMER_HZ нужен в расчёте периода звука. Для реального подключения звука на стенде должен быть замкнут SA2.7: это указано в README проекта.

clock.c: где реально настраивается частота

Core/Src/clock.c, строки 24-31

```
24      osc.PLL.PLLState = RCC_PLL_ON;
25      osc.PLL.PLLSource = RCC_PLLSOURCE_HSI;
26      osc.PLL.PLLM = 8;
27      osc.PLL.PLLN = 120;
28      osc.PLL.PLLP = RCC_PLLP_DIV2;
29      osc.PLL.PLLQ = 5;
30      if (HAL_RCC_OscConfig(&osc) != HAL_OK) return false;
31      clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
```

Здесь задаётся PLL: из исходных 16 МГц получается **120 МГц** для контроллера. Числа 8, 120 и делитель 2 стоят прямо в этом фрагменте. `HAL_RCC_OscConfig` применяет настройку, затем выбирается источник PLL.

Core/Src/clock.c, строки 19-21

```
19      clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
20      clk.APB1CLKDivider = RCC_HCLK_DIV4;
21      clk.APB2CLKDivider = RCC_HCLK_DIV2;
```

Что настраивается	Частота после clock_init
Ядро и AHB	120 МГц.
APB1	30 МГц; для него выбран делитель 4.
APB2	60 МГц; для него выбран делитель 2.
Таймер TIM2	60 МГц. Это значение использует `sound_period_ticks`.

В строках **33-34** фиксируется настройка таймеров TIMPRE и применяется итоговая конфигурация. В начале функции, 14-22, есть предварительный переход на HSI для корректного запуска после отладчика.

Где менять: реальные частоты задаёт `clock.c`. Значения в `board.h` должны им соответствовать; одна правка `BOARD_SYSCLK_HZ` сама не перенастроит контроллер.

6. console.c: ввод клавиш и ответы

В Console хранятся rx_epoch - номер потери входа и reported_scale_note - номер показанной ноты гаммы (Core/Inc/console.h, 6). Клавиши обрабатываются сразу.

Где символ передаётся в piano

Core/Src/console.c, строки 72-75

```
72          (void)serial_write(&byte, 1);
73          piano_event_t event = piano_input(piano, byte, now_ms);
74          respond(event, piano, byte);
75          if (event == PIANO_EVENT_SCALE) console->reported_scale_note = PIANO_NO_NOTE;
```

69: взять байт из RX. **70-71:** убедиться, что при чтении не произошла потеря входа. **72:** вернуть символ в терминал - эхо. **73:** выполнить действие клавиши. **74:** сформировать ответ по результату.

За сообщения отвечает respond

Событие	Что выводится; строки в console.c
PIANO_EVENT_NOTE	Имя ноты, октава, частота и длительность: 20-27.
PIANO_EVENT_OCTAVE PIANO_EVENT_DURATION	Новые выбранные настройки: 29-40.
PIANO_EVENT_SCALE	Сообщение о запуске гаммы: 41-46.
PIANO_EVENT_LIMIT	LIMIT и допустимые границы настроек: 47-49.
PIANO_EVENT_INVALID	Ошибка и код неподходящего символа: 50-52.

Где показываются новые ноты гаммы

Core/Src/console.c, строки 77-82

```
77      if (!piano_is_scale(piano)) console->reported_scale_note = PIANO_NO_NOTE;
78      else if (console->reported_scale_note != piano_current_note(piano) &&
79               serial_tx_free() >= CONSOLE_REPLY_RESERVE) {
80          respond(PIANO_EVENT_NOTE, piano, 0);
81          console->reported_scale_note = piano_current_note(piano);
82      }
```

Если играет гамма и текущая нота ещё не сообщалась, консоль выводит её при наличии места в TX. reported_scale_note запоминает показанный номер, чтобы одно сообщение не повторялось каждый проход цикла.

Запас для ответа и потеря ввода

Строка **60** берёт до 16 байт за вызов и проверяет запас под полный ответ: 192 байта, заданные в console.h, строка 5. Строки **61-66** замечают изменение rx_epoch и просят повторить последнюю клавишу. Воспроизведение продолжает обновляться в main.

7. serial.c и uart_hal.c: доставка клавиш

RX - приём, TX - отправка. Массивы очередей объявлены в `serial.c`, строка 5. Их размеры 128 и 512 байт заданы в `Core/Inc/serial.h`, строки 6-11.

Ключевые функции serial.c

Функция и строки	Что делает
`serial_received`, 53-65	Кладёт принятый байт в RX.
`serial_read`, 35-45	Отдаёт консоли самый ранний байт из RX.
`serial_write`, 21-34	Ставит весь текст в TX. Если места мало, возвращает `false`, ничего не добавив.
`serial_service`, 87-118	Продвигает отправку. В этой работе используется ветка прерываний.
`serial_transmitted`, 74	Сообщает, что текущий байт отправлен: сбрасывает `busy`.

Как UART отдаёт принятый байт в очередь

Core/Src/uart_hal.c, строки 68-70

```
68      if (handle->ErrorCode == HAL_UART_ERROR_NONE) {
69          serial_received(rx_byte);
70          if (irq_enabled) (void)HAL_UART_Receive_IT(handle, &rx_byte, 1);
```

Обработчик `USART1_IRQHandler` (64) вызывает HAL. После приёма HAL вызывает `HAL_UART_RxCpltCallback` (65-72): байт попадает в `serial_received`, и сразу запускается приём следующего байта.

Как serial_service запускает отправку

Core/Src/serial.c, строки 100-107

```
100     if (!tx_count || busy) return;
101     if (irq_mode) {
102         uint32_t p = lock();
103         tx_byte = tx[tx_tail];
104         busy = true;
105         if (uart_backend_transmit(&tx_byte, true)) {
106             tx_tail = (uint16_t)((tx_tail + 1U) % SERIAL_TX_CAPACITY);
107             --tx_count;
```

Если TX пуста или байт ещё занят, функция возвращается. Иначе берёт байт, ставит `busy = true` и запускает передачу. При успехе сдвигает очередь. Завершение через HAL-обработчик 73-76 в `uart_hal.c` вызывает `serial_transmitted`.

Настройки USART1 и скорости - `uart_hal.c`, 10-25. PA9/PA10 с AF7 и освобождение общих линий PC6/PC7 - `uart_pins.c`, 3-13. При ошибке или переполнении RX меняется `rx_epoch`: `serial.c`, 55-72.

8. Остальные файлы и запуск проекта

Файл или группа	За что отвечает
`Core/Src/gpio_driver.c` `Core/Inc/gpio_driver.h`	Включение порта - 12-19; настройка выводов - 45-96. Его вызывают `sound_init` и `uart_pins_init`.

`Core/Startup/`
`Core/Src/system_stm32f4xx.c`
`Drivers/`
	Стартовый и системный код, CMSIS и HAL производителя. В стартовом файле вызов `main` находится в строке 100.

`Core/Inc/`
`stm32f4xx_hal_conf.h`
`STM32F427VGTX_FLASH.ld`
	Настройки библиотеки и правила размещения программы и данных в памяти.

`tools/build.py`	Собирает приложение; с `--test` собирает отдельную тестовую программу.
`tools/board.py` `SDK11M.cfg`	Прошивка, проверка подключения и отладка через FT2232/JTAG. По умолчанию используется `build/lab3.elf`.

`tests/test_lab3.c`
`tests/run_tests.py`
`tests/test.ld`
	Проверки нот, времени, таймера, UART и запуска программы. Python выполняет ARM-код в эмуляторе.

`.project`, `.cproject` `.gitignore`	Настройки проекта и сборки в IDE; список файлов, которые Git не должен добавлять.
`README.md`, `docs/`	Описание управления, схема стенда, пояснения к защите и протокол проверок.

Команды из корня репозитория

```
python3 lab3/tools/build.py
python3 lab3/tools/board.py flash
```

Сборка скриптом создаёт `lab3/build/lab3.elf` и `.bin`. Сборка в CubeIDE кладёт файлы в `Debug` или `Release`. Для неё скрипту прошивки передаётся путь через `--elf`, например `lab3/Debug/lab3.elf`.

Где найти примеры проверок кода

В `tests/test_lab3.c` функция `test_entry` начинается на строке **41**: проверяет исходные настройки, клавиши, границы, гамму и параметры звука. `test_step` (24-30) повторяет действия главного цикла.

В `tests/run_tests.py`: частоты и таймер - `frequencies_and_timer` (186), связь и клавиши - `uart_application` (207), запуск готовой прошивки - `boot_firmware` (287).

9. Что показать на защите

Один пример: нажали 6 после запуска

1. UART принимает символ 6; через RX и `console_update` он попадает в `piano_input`.
2. Строка 71 в `piano.c` превращает 6 в номер ноты 5 - это ля.
3. `start_playback` сохраняет начальные настройки: октава 5/9, длительность 1000 мс. Частота из таблицы равна 440000 миллигерц, то есть 440 Гц.
4. `main` передаёт её в `sound_set_frequency`. Для TIM2 получаются `ticks = 136364`, `ARR = 136363`, `CCR1 = 68182`: звук начинается.
5. Через 1000 мс `piano_update` завершает ноту. Текущая частота становится 0, и `sound` выключает сигнал.

Быстрая подсказка: где что найти

Нужно показать или изменить	Открой
Порядок работы программы	`Core/Src/main.c`, 19-36
Начальную октаву, длительность и границы	`Core/Inc/piano.h`, 11-20
Обработку клавиш	`Core/Src/piano.c`, 59-107
Сохранение настроек текущей ноты/гаммы	`Core/Src/piano.c`, 33-42
Длительность ноты и ход гаммы	`Core/Src/piano.c`, 109-133
Частоты нот / выбор частоты из таблицы	`Core/Src/piano.c`, 5-15, 177-184
Параметры таймера / выключение звука	`Core/Src/sound.c`, 48-66 / 37-46
Реальные настройки частот / вывод звука	`Core/Src/clock.c`, 4-35 `Core/Inc/board.h`, 5-10
Приём клавиши и ответы терминалу	`Core/Src/console.c`, 15-83

Как коротко объяснить своё решение

«В `piano` я храню выбранные настройки и состояние воспроизведения. Консоль передаёт нажатия клавиш, а главный цикл проверяет время. Звуковой модуль получает частоту текущей ноты и настраивает таймер. Когда длительность заканчивается, нота меняется или звук выключается».

Памятка составлена по текущим исходникам `lab3`. Фрагменты взяты из файлов автоматически; исходный код при подготовке документа не менялся.